Manual de resolución: Ejercicios 1 a 5

Guía paso a paso con código en Python (y R donde se pide) para resolver los cinco ejercicios de la hoja.

Preparación general

Instala las librerías una sola vez:

```
pip install pandas numpy scikit-learn statsmodels xlrd openpyxl
```

`xlrd` es necesaria para leer archivos `.xls` con pandas.

Ejercicio 1: Punto de equilibrio

Datos: precio de venta = 40 USD, costo por camisa = 24 USD, comisión = 2 USD por camisa, gastos fijos = 3,500 USD.

Idea clave: el punto de equilibrio es la cantidad de camisas donde la utilidad es cero.

- Margen de contribución por camisa = precio − costo − comisión = 40 − 24 − 2 = **14 USD**
- Punto de equilibrio = gastos fijos ÷ margen de contribución = 3,500 ÷ 14 = **250 unidades**

En hoja de cálculo (Excel / Google Sheets)

Celda	Concepto	Valor o fórmula
B1	Precio	40
B2	Costo	24
B3	Comisión	2
B4	Gastos fijos	3500
B5	Margen de contribución	`=B1-B2-B3`
B6	Punto de equilibrio	`=B4/B5`

En Python

```
precio = 40
costo = 24
comision = 2
fijos = 3500

margen = precio - costo - comision
punto_equilibrio = fijos / margen
print(f"Punto de equilibrio: {punto_equilibrio:.0f} unidades")
```

Resultado: 250 unidades (coincide con tu respuesta).

Comprobación: ingresos = 250 × 40 = 10,000; costos = 250 × 26 + 3,500 = 10,000. Utilidad = 0.

Ejercicio 2: Métricas de los sexenios (Python y R)

Se piden media, varianza, desviación estándar, máximo, mínimo y rango del tipo de cambio de cada sexenio. Debes resolverlo **dos veces**: con Python y con R.

Nota: los archivos `sexenio-1.xls` y `sexenio-2.xls` no venían en las fotos, así que el código usa un nombre de columna genérico. Abre tu archivo y cambia `"Tipo de cambio"` por el nombre real de la columna.

Pasos

1. Coloca los dos archivos `.xls` en la misma carpeta que tu script.
2. Lee cada archivo.
3. Selecciona la columna del tipo de cambio.
4. Calcula las seis métricas.
5. Llena la tabla de la hoja.

En Python

```
import pandas as pd

def metricas(archivo, columna):
    df = pd.read_excel(archivo)
    x = pd.to_numeric(df[columna], errors="coerce").dropna()
    return {
        "Media": x.mean(),
        "Varianza": x.var(),          # muestral (n-1)
        "Desviación estándar": x.std(),
        "Máximo": x.max(),
        "Mínimo": x.min(),
        "Rango": x.max() - x.min(),
    }

col = "Tipo de cambio"   # cámbialo por el nombre real
tabla = pd.DataFrame({
    "Sexenio 1": metricas("sexenio-1.xls", col),
    "Sexenio 2": metricas("sexenio-2.xls", col),
})
print(tabla.round(4))
```

En R

```
library(readxl)

metricas <- function(archivo, columna) {
  df <- read_excel(archivo)
  x <- as.numeric(df[[columna]])
  x <- x[!is.na(x)]
  c(Media = mean(x),
    Varianza = var(x),        # muestral (n-1)
    Desv_estandar = sd(x),
```

```
    Maximo = max(x),
    Minimo = min(x),
    Rango = max(x) - min(x))
}

col <- "Tipo de cambio"   # cámbialo por el nombre real
tabla <- data.frame(
  Sexenio_1 = metricas("sexenio-1.xls", col),
  Sexenio_2 = metricas("sexenio-2.xls", col)
)
print(round(tabla, 4))
```

Puntos a cuidar

- Python (pandas) y R usan por defecto la varianza **muestral** (divide entre n−1), así que los resultados deben coincidir. NumPy, en cambio, usa la poblacional por defecto.
- Si el archivo trae filas de encabezado extra, usa `skiprows=` en pandas o `skip =` en R.
- Si la columna tiene texto o celdas vacías, la conversión numérica y `dropna()` las limpian.
- Si mandas los archivos, puedo calcular la tabla completa.Ejercicio 3: Tres regresiones simples

Objetivo: predecir el precio de venta con **un factor a la vez**: pies cuadrados, dormitorios y antigüedad.

Pasos

1. Captura los 17 datos de la tabla.
2. Ajusta tres modelos: Precio ~ pies², Precio ~ dormitorios, Precio ~ antigüedad.
3. Anota intercepto, pendiente y R^2 de cada uno.

Código

```
import numpy as np
import statsmodels.api as sm

precio = np.array([64000,59000,61500,79000,87500,92500,95000,113000,115000,
                   138000,142500,144000,145000,147500,144000,155500,165000])
pies   = np.array([1670,1339,1712,1840,2300,2234,2311,2377,2736,
                   2500,2500,2479,2400,3124,2500,4062,2854])
dorm   = np.array([2,2,3,3,3,3,3,3,4,3,4,3,3,4,3,4,3])
anti   = np.array([30,25,30,40,18,30,19,7,10,1,3,3,1,0,2,10,3])

for nombre, x in [("Pies cuadrados", pies), ("Dormitorios", dorm), ("Antigüedad",
anti)]:
    modelo = sm.OLS(precio, sm.add_constant(x)).fit()
    print(nombre, "-> intercepto, pendiente:", modelo.params.round(4),
          "R2:", round(modelo.rsquared, 4))
```

Resultados esperados

Modelo	Ecuación	R^2
Pies cuadrados	Precio = 2,367.26 + 46.60 × pies²	0.646
Dormitorios	Precio = 1,923.47 + 36,137.76 × dormitorios	0.365

Modelo	Ecuación	R^2
Antigüedad	Precio = 147,670.87 − 2,424.16 × años	0.777

Cómo interpretar

- **Pies cuadrados:** cada pie cuadrado adicional suma aproximadamente 46.60 USD al precio.
- **Dormitorios:** cada dormitorio extra suma aproximadamente 36,138 USD.
- **Antigüedad:** cada año adicional resta aproximadamente 2,424 USD. Es el mejor predictor individual (R^2 más alto).

Revisa tus anotaciones: en la foto escribiste 880.28953, 1.96e+00, 50.3638181 y −0.0003204. Esos números no coinciden con los coeficientes de arriba. Probablemente sean otras estadísticas de la salida (errores estándar, valores p, etc.). Para el ejercicio debes reportar intercepto y pendiente de cada modelo.

Ejercicio 4: Regresión múltiple con pies² y dormitorios

Objetivo: modelo Precio ~ pies² + dormitorios, y predecir una casa de 2,000 pies² con 3 dormitorios.

Pasos

1. Arma una matriz con dos variables explicativas.
2. Ajusta la regresión múltiple.
3. Sustituye los valores en la ecuación.

Código

```
X = sm.add_constant(np.column_stack([pies, dorm]))
modelo = sm.OLS(precio, X).fit()
print(modelo.params)      # [intercepto, b_pies, b_dorm]
print("R2:", modelo.rsquared)

prediccion = modelo.predict([[1, 2000, 3]])
print(prediccion)
```

Resultado

Precio = 5,701.45 + 48.505 × pies² − 2,540.39 × dormitorios (R^2 = 0.646)

Para 2,000 pies² y 3 dormitorios:

5,701.45 + 48.505 × 2,000 − 2,540.39 × 3 = **95,091.18 USD**

Esto coincide con tu respuesta.

Observación: al agregar dormitorios, el R^2 casi no mejora (0.6457 → 0.6465) y el coeficiente de dormitorios sale negativo. Eso indica que los dormitorios aportan poco una vez que ya se considera el tamaño.

Ejercicio 5: Regresión múltiple con pies², dormitorios y antigüedad

Objetivo: modelo Precio ~ pies² + dormitorios + antigüedad, y predecir una casa de 10 años, 2,000 pies² y 3 dormitorios.

Código

```
X = sm.add_constant(np.column_stack([pies, dorm, anti]))
modelo = sm.OLS(precio, X).fit()
print(modelo.params)      # [intercepto, b_pies, b_dorm, b_anti]
print("R2:", modelo.rsquared)

prediccion = modelo.predict([[1, 2000, 3, 10]])
print(prediccion)
```

Resultado

Precio = 82,185.65 + 25.94 × pies² − 2,151.74 × dormitorios − 1,711.54 × antigüedad (R^2 = 0.886)

Sustituyendo (2,000 pies², 3 dormitorios, 10 años):

82,185.65 + 25.94 × 2,000 − 2,151.74 × 3 − 1,711.54 × 10 = **110,496.58 USD**

Esto coincide con tu respuesta (110,496.6).

Observación: el R^2 sube a 0.886, el mejor de todos los modelos. Combinar tamaño y antigüedad explica mucho mejor el precio.

Resumen de resultados

Ejercicio	Resultado
1	Punto de equilibrio: 250 unidades
2	Tabla de métricas por sexenio (requiere los archivos .xls)
3	Tres regresiones simples (R^2: 0.646, 0.365, 0.777)
4	Predicción: 95,091.18 USD
5	Predicción: 110,496.58 USD